

Rapport de sécurité — Folio+

Date : 15/07/2026 (mis à jour le 16/07/2026 — correctifs des points moyens et faibles) • **Périmètre** : revue manuelle du repo (migrations SQL/RLS, Edge Functions, GRANT/REVOKE, politiques storage, dépendances npm, code client) + advisor sécurité Supabase + **scan complet de l'historique git** (`git log --all -p` , pas seulement l'état courant).

Méthode : lecture de code + introspection live de la base (`pg_policies` , `pg_proc` , `pg_trigger` , `has_function_privilege` , `vault.decrypted_secrets`) sur le projet Supabase `rctedezgdxadmjkjeawsj` .

Résumé

Gravité	Nombre	Résumé
✅ Corrigé	1	Secret <code>CRON_SYNC_SECRET</code> commité en clair dans l'historique git (poussé sur GitHub), tournait toujours avec la valeur d'origine
✅ Corrigé / 🛑 Bloqué	2	Injection HTML dans les emails transactionnels (corrigé) ; protection "mot de passe compromis" désactivée (bloqué, plan Pro requis)
✅ Corrigé / ℹ️ Laissé tel quel	3	Anti-abus (rate limit) sur le formulaire de contact ; comparaison de secret en temps constant (corrigés) ; garde-fou anti-auto-promotion admin sur un seul trigger (laissé tel quel, volontairement)

RLS activée sur toutes les tables, GRANT/REVOKE cohérents, politiques storage correctes, `npm audit` propre. Tous les points corrigeables sont corrigés et vérifiés ; seule la protection mot de passe compromis reste bloquée par le plan Supabase actuel (Free), et le garde-fou sur ``role`` est resté en l'état par choix (raisonnement détaillé plus bas).

Critique 0. Secret `CRON_SYNC_SECRET` commité en clair, toujours actif

✅ Corrigé le 15/07

Description

Un scan de l'historique complet (`git log --all -p` , et pas seulement l'état courant des fichiers) révèle que `CRON_SYNC_SECRET` — le secret qui authentifie le chemin cron de `supabase/functions/sync-notion-veille/index.ts` — a été codé en dur dans le commit `d450e43` (13/07, "feat(veille) : integre la Veille Design Hebdo Notion -> Supabase") :

```
const CRON_SYNC_SECRET = "11ec190c68608885e91b50e8773ecde2732c1ee25aadb2fc1ee33c575d2c86b8";
```

Un commit suivant (`c43fbea` , même jour, "fix(veille) : deplace CRON_SYNC_SECRET en secret Edge Function, plus de hardcoded") a bien déplacé la lecture vers `Deno.env.get("CRON_SYNC_SECRET")` — **mais la valeur elle-même n'a jamais été changée**. Vérifié directement en base (`vault.decrypted_secrets`) : le secret `cron_sync_secret` actuellement stocké en Vault est **exactement cette même valeur littérale**.

Le commit `d450e43` est bien présent sur `origin/main` et `origin/feat/veille-design-hebdo-synchronisation` — donc poussé sur GitHub depuis le 13/07.

Effet

N'importe qui ayant (ou ayant eu) accès au repo GitHub privé dispose de ce secret et peut appeler `sync-notion-veille` en contournant entièrement le contrôle "admin only" :

```
curl -X POST https://<projet>.functions.supabase.co/sync-notion-veille -H "x-cron-secret: 11ec19"
```

Le repo étant privé, l'exposition reste limitée à qui a (eu) accès au repo — mais un secret ayant transité par l'historique git doit être considéré comme compromis, indépendamment de la visibilité du repo.

✓ Correctif appliqué

- Nouvelle valeur aléatoire (256 bits) générée pour `cron_sync_secret` .
- Mise à jour dans Supabase Vault via `vault.update_secret()` (le job `pg_cron` lit la valeur dynamiquement à chaque exécution via `vault.decrypted_secrets` , aucune modification de la migration nécessaire).
- Mise à jour du secret de l'Edge Function `sync-notion-veille` (`CRON_SYNC_SECRET` , Dashboard Supabase → Edge Functions → Secrets).

Vérifié par appel direct à la fonction déployée :

- Ancienne valeur (celle qui avait fuité) → `403 forbidden` (rejetée, tombe sur le chemin "pas admin").
- Nouvelle valeur → `200 OK` , synchro fonctionnelle.

La rotation est complète et confirmée des deux côtés (Vault + Edge Function synchronisés).

Moyenne

1. Injection HTML dans les emails transactionnels

✓ Corrigé le 16/07

Description

Dans `supabase/functions/webhook-contact-created/index.ts` et `supabase/functions/webhook-access-request-created/index.ts` , des champs contrôlés par un visiteur anonyme (`name` / `message` du formulaire de contact, `full_name` à l'inscription) sont interpolés **sans échappement** dans le HTML des emails envoyés à l'admin :

```
// webhook-contact-created/index.ts:82-83
<p><strong>${record.name || ""}</strong> (${record.email}) a envoyé un message :</p>
<blockquote>${record.message || ""}</blockquote>
```

```
// webhook-access-request-created/index.ts:88
<p>${visitor?.full_name || visitor?.email || "Un visiteur"} demande l'accès au projet <strong>${
```

Effet

N'importe qui peut soumettre le formulaire public (ou créer un compte) avec

`name / message / full_name` contenant du HTML arbitraire : liens de phishing déguisés, images de tracking (fuite d'IP/read-receipt), mise en page cassée. Pas d'exécution JS classique (la plupart des clients mail retirent `<script>`), mais c'est une injection HTML stockée qui atterrit dans la boîte mail de l'admin.

✅ Correctif appliqué

Ajout d'une fonction `escapeHtml()` locale (même pattern dupliqué que le reste du code de ces fonctions, pas de module partagé entre Edge Functions Deno) dans les 4 fonctions `webhook-*`, appliquée à tous les champs utilisateur interpolés dans le HTML (`name / email / message` du contact, `full_name / rejection_reason` des demandes d'accès, `title` des projets par défense en profondeur même si admin-only).

Vérifié en production : appel direct de `webhook-contact-created` avec une charge utile contenant `<img src=x onerror=alert(1)>` et `<script>alert(1)</script>` dans `name / message`. Email réellement envoyé et inspecté via l'API Resend (`get-email`) — le HTML reçu contient `<img src=x onerror=alert(1)>` et `<script>...</script>` en texte littéral, plus aucune balise active. Donnée de test supprimée après coup.

Moyenne 2. Protection "mot de passe compromis" désactivée

🚫 Bloqué (fonctionnalité Pro)

Description

L'advisor sécurité Supabase (`auth_leaked_password_protection`) signale que la vérification contre HavelBeenPwned.org est désactivée sur l'auth.

Effet

Le flux `AccessRequestModal` crée de vrais comptes avec un mot de passe choisi par un inconnu (self-service, F-12) — sans cette protection, rien n'empêche l'usage d'un mot de passe déjà compromis dans une fuite connue.

Statut : non applicable pour l'instant

Vérifié dans le Dashboard (Authentication → Attack Protection) : "Prevent use of leaked passwords" est bien la bonne option, mais elle est réservée aux **projets Supabase sur plan Pro** — pas de toggle disponible sur le plan actuel (Free). Aucune action possible côté code ou configuration tant que le projet reste sur ce plan. À reconsidérer si/quand le projet passe sur un plan payant.

📄 Laissé tel quel

3. Garde-fou anti-auto-promotion admin reposant sur un seul trigger

Description

La policy RLS `user_profiles_update_own` (`WITH CHECK (id = auth.uid())`) ne restreint pas la colonne `role` — à elle seule, elle laisserait n'importe quel utilisateur authentifié s'attribuer `role = 'admin'` via un `UPDATE` direct. Ce qui bloque réellement cette escalade aujourd'hui est le trigger `trg_prevent_role_self_update` (`BEFORE UPDATE OF role ON user_profiles` , fonction `prevent_role_self_update()`), qui lève une exception si un non-admin tente de changer son propre `role` .

Effet

Aucun aujourd'hui — le trigger fonctionne et a été vérifié (`SECURITY DEFINER` , logique correcte : compare `NEW.role / OLD.role` , vérifie `auth.uid() = OLD.id` , vérifie le rôle courant via une sous-requête). Mais c'est un **point de défaillance unique** : si ce trigger est un jour supprimé ou modifié par inadvertance (ex. nettoyage futur de migrations) sans que la dépendance soit connue, l'escalade de privilèges redevient possible via la seule RLS.

Pourquoi ce n'est pas corrigé en RLS

Tenté puis écarté : dupliquer cette vérification dans le `WITH CHECK` de la policy (comparer `role` à sa valeur avant modification via une sous-requête sur `user_profiles`) se heurte exactement au piège rencontré et corrigé au point #4 — une sous-requête corrélée référençant la même table ne peut pas distinguer sans ambiguïté "l'ancienne valeur" de "la nouvelle" en scope SQL simple, contrairement à un trigger `BEFORE UPDATE` qui a un accès natif et non ambigu à `OLD / NEW` . Forcer une solution basée sur les policies serait soit redondant et fragile, soit risquerait de casser silencieusement (comme documenté au point #4) la capacité d'un admin à modifier le rôle d'un autre utilisateur (Supabase n'a que 2 rôles Postgres réels — `anon / authenticated` — la distinction "admin" est une donnée applicative vérifiée par `get_my_role()` , pas un rôle Postgres séparé, donc impossible de restreindre la colonne par un `GRANT` ciblé sans bloquer aussi les admins).

Recommandation

Le trigger est le bon mécanisme pour ce cas précis — **conservé tel quel**. Documenté explicitement dans `CLAUDE.md` (section RLS — pièges à ne pas reproduire) pour qu'une future session ne le supprime jamais sans réaliser qu'il est la seule protection sur `role` .

Faible

4. Absence de limitation anti-abus sur les endpoints publics

✓ Corrigé le 16/07

Description

`contacts_insert_anyone` (formulaire de contact) était ouvert à tout le monde, sans CAPTCHA ni throttling. `AccessRequestModal` (création de compte) reste hors scope de ce correctif : elle nécessite un vrai `signUp()` , déjà soumis au rate limiting natif de Supabase Auth.

Effet

Un script pouvait spammer le formulaire de contact : coût en quota Resend, boîte mail admin encombrée.

✅ Correctif appliqué

Policy `contacts_insert_anyone` limitée à 3 soumissions par email par tranche de 10 minutes (migrations `20260716090000_contacts_insert_rate_limit.sql` puis `20260716090500_fix_contacts_insert_rate_limit.sql`). Ne remplace pas un CAPTCHA (n'empêche pas la rotation d'emails jetables), mais coupe le spam trivial d'un même email en boucle, sans dépendance à un service tiers.

Bug rencontré puis corrigé pendant l'implémentation : la 1ère version comparait `c.email = email` dans une sous-requête corrélée sur la table `contacts` elle-même — la portée SQL résout `email` vers l'alias le plus proche (`c`), donnant silencieusement `c.email = c.email` (toujours vrai, pas d'erreur). Vérifié en pratique : 4 soumissions rapides du même email passaient toutes (201×4) avant correctif. Corrigé en extrayant le comptage dans une fonction `SECURITY DEFINER` prenant l'email en paramètre (`contacts_recent_count`), qui lève l'ambiguïté. **Re-vérifié** : 3 soumissions passent, la 4^e est rejetée (42501), un email différent n'est pas affecté par la limite d'un autre. Données de test supprimées après coup.

Faible 5. Comparaison du secret webhook non "constant-time"

✅ Corrigé le 16/07

Description

Dans les 4 fonctions `webhook-*` et `sync-notion-veille`, la vérification du secret partagé utilisait une comparaison de chaînes standard (`provided === expected`), potentiellement sensible à une attaque par timing (deviner le secret octet par octet en mesurant les différences de latence).

Effet

Risque pratique quasi nul pour une cible à faible valeur comme celle-ci (le jitter réseau domine largement toute différence de timing mesurable), mais coût de correction quasi nul également.

✅ Correctif appliqué

Ajout d'une fonction `timingSafeEqual()` locale (comparaison XOR octet par octet, sans retour anticipé sur la première différence) dans les 5 fonctions concernées, déployées. **Vérifié** : `sync-notion-veille` et `webhook-contact-created` répondent toujours correctement avec le bon secret (200) et rejettent toujours un mauvais secret (403/401) après le changement.

Vérifié et sain (pas d'action nécessaire)

- **RLS** activée sur toutes les tables `public.*`.

- **GRANT/REVOKE** cohérents sur les fonctions sensibles : `dispatch_webhook` , `get_webhook_dispatch_secret` , `handle_new_user` , `prevent_role_self_update` , `create_access_request_notifications` — non exécutables par `anon` / `authenticated` (vérifié via `has_function_privilege`).
`get_my_role` / `get_public_cal_username` / `project_deletion_status` sont intentionnellement publiques et ne fuient aucune donnée au-delà de ce qui est déjà public (vérifié le contenu de chaque fonction).
- **soft_delete_project** : callable par `anon` / `authenticated` au niveau GRANT, mais protégée en pratique car `SECURITY INVOKER` (pas `DEFINER`) — les `UPDATE` / `DELETE` internes sont filtrés par RLS selon l'appelant, et la fonction vérifie explicitement qu'une ligne a bien été affectée avant de retourner un succès.
- **Politiques storage** (`project-thumbnails` , `designer-photos`) : écriture admin-only, pas de policy `SELECT` superflue (pas de listing public possible).
- **anonymize-rgpd** : la cible est toujours résolue depuis le JWT de l'appelant (`auth.getUser()`), jamais depuis le corps de la requête — un utilisateur ne peut anonymiser que son propre compte.
- **.env** : jamais tracké actuellement (`git ls-files` vide), bien couvert par `.gitignore` (`.env` + `.env.*` , exception uniquement pour `.env.example`). `.env.admin` (service role key, utilisé par `scripts/`) n'a **jamais** été commité, à aucun moment de l'historique.
- **Séparation clé publique/secrète** : `src/integrations/supabase/client.ts` ne charge que `VITE_SUPABASE_PUBLISHABLE_KEY` (`sb_publishable_...`) côté client — aucune occurrence de `sb_secret_` ou de la service role key dans le bundle, actuellement ou dans l'historique.
- **Historique git — nuance** : un `.env` initial (commit `3dcab73` , 06/07) a bien été commité puis supprimé, mais ne contenait qu'une clé **publiable** (`sb_publishable_...` , faite pour être publique) et l'URL du projet Lovable Cloud d'origine — sans risque. Le scan complet de l'historique a en revanche révélé le secret `CRON_SYNC_SECRET` en clair (cf. point critique #0 ci-dessus) — c'est le seul vrai secret trouvé, mais il est réel et toujours actif.
- **Dépendances** : `npm audit --production` → 0 vulnérabilité connue.
- **XSS côté client** : aucun `dangerouslySetInnerHTML` actif (seule occurrence dans le code mort `src/components/ui/chart.tsx` , jamais importé) ; `MarkdownContent.tsx` (react-markdown) n'utilise aucun plugin de rendu HTML brut (pas de `rehype-raw`).